



Sources: read these before reverse-engineering anything

This document exists because the first pass of this project derived the `song.ini` key table from YARG.Core's source when **the official wiki documents it better**, and got several things wrong as a result. The corrections are listed at the bottom. Check this page before deriving a format from source.

Primary documentation — check first

Source	Covers	Quality
YARG Wiki — <code>song.ini</code>	The full tag table with types, meanings, enum values, deprecated aliases, defaults, and a compatibility column saying whether YARG acts on a key today or merely parses it	Better than the source. The compatibility tiers and the documented defaults are not derivable from <code>SONG_INI_OUTLINES</code> at all
Guitar Game Chart Formats (TheNathannator)	<code>.chart</code> and <code>.mid</code> — both marked complete — plus <code>song.ini</code> . Audio conventions are a TODO	The reference for chart notation. Read this before Phase 1 step 6 ; do not reverse-engineer the MIDI preparers
YARG Wiki — Songs	CON, ex-CON and ex-PKG layouts, <code>songs_updates</code>	Good for the RB side, which we deliberately do not support. <code>.chart</code> and <code>.mid</code> sections are stubs
YARG Wiki — <code>notes.mid</code>, <code>notes.chart</code>	Chart notation	Check before step 6
mdsitton/SngFileFormat	The <code>.sng</code> container spec and the reference <code>SngCli</code> encoder	The only spec for <code>.sng</code>

Source	Covers	Quality
YARG CONTRIBUTING.md	The six-tier scope framework, including what will be rejected on sight	Authoritative on what upstream will accept

What is genuinely NOT documented

Derived from source and from measurement, because nothing else covers it:

- **.sng has no wiki page at all.** The Songs page lists [.chart](#), [.mid](#), CON and ex-PKG, and does not mention [.sng](#). The mdsitton spec is the only description of the container.
- **The song cache** ([songcache.bin](#)) — undocumented anywhere.
- **Song identity.** That a song is [SHA1\(chart file bytes\)](#) and nothing else appears in no documentation; it came from [HashWrapper](#) and [ScanChart](#).
- **Audio stem naming** — TheNathannator lists audio as a TODO, and the clean/explicit variants are a YARG-specific extension no third-party document covers.
- **Scanner behaviour** — that a headerless [song.ini](#) yields no metadata, that a chart with no audio is rejected, and that a folder with no [song.ini](#) is silently skipped were all established by scanning a corpus with the game and reading its own report. See [TEST-CORPUS.md](#).

What reading the wiki late cost

Four real defects, all shipped and then fixed:

1. [album_track](#) / [playlist_track](#) **default to 16000, not 0.** We used 0. The difference is visible to a player: unnumbered songs sort to the *front* of an album instead of the end.
2. [track](#) **is a deprecated alias for [album_track](#), and [album_track](#) wins outright** when both are present. We fell back to [track](#) whenever [album_track](#) was zero — which is exactly wrong for a song explicitly numbered 0.
3. [frets](#) **is a deprecated alias for [charter](#)**. It was in our key table but mapped to nothing, so a chart using the old name lost its charter credit.
4. **Eleven keys were missing entirely** — [year_recorded](#), [year_released](#), [parts_vocals_harm](#), [diff_guitar_coop_real](#), [diff_guitar_coop_real_22](#), [diff_rhythm_real](#), [diff_rhythm_real_22](#), [link_newgrounds](#), [link_soundcloud](#), [link_tiktok](#).

Plus two things the wiki gives that the source cannot: the **compatibility tier** (a key can be parsed today but acted on only in some future version — presenting those as meaningful promises behaviour the client does not have), and the **meanings of the [rating](#) and [vocal_gender](#) enums**.

One unresolved conflict

The wiki documents `vocal_gender` as an **integer** enum (0=Female ... 4=Unspecified). YARG.Core parses the key as a **String**. Both cannot be right about the on-disk shape. The scanner stores the raw value and normalises nothing, because converting either way would be inventing a fact. Settle it by reading the modifier's conversion before anything depends on the value.
